Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

VIT Chennai

School of Computer Science and Engineering (SCOPE)

B.Tech Computer Science and Engineering

TalentCloud

AWS Cloud Deployment and Architecture

Review 2 — Project Report

Course Code	BCSE355L
Course Title	AWS Solutions Architect
Faculty	Sellam V
Class Slot	E2 + TE2
Team Member 1	Ayushi Das — 24BCE1438
Team Member 2	Bahaar Boral — 24BCE1760
Submission Date	22 October 2026

Confidential — Academic Submission Only

Table of Contents

2. Abstract **4**
3. Introduction **4**
3.1 Background and Problem Statement 4
3.2 Intended Users 4
3.3 Objectives 5
3.4 Scope and Exclusions 5
3.5 Requirements Summary 6
4. Methodology **6**
4.1 Proposed Architecture 6
4.2 AWS Services and Design Justification 8
4.3 Implementation Procedure 9
4.4 Security, Availability, and Cost Considerations 11
4.4.1 Security Analysis and Network Restrictions 11
4.4.2 Availability, Resilience, and Backup Limitations 12
4.4.3 AWS Cost Estimation (Pricing Calculator Analysis) 12
4.5 Testing and Current Progress 13
5. Conclusion **19**
6. References **20**
Appendix A: Key Deployment Commands **21**
Appendix B: Confirmed docker-compose.yml Service Definitions **21**
Appendix C: Prisma Schema — Key Database Models **22**
Appendix D: Team Member Contributions and Role Allocation **22**

List of Figures

Figure 1. TalentCloud Review 2 deployment architecture (one EC2 host, Docker Compose) 7
Figure 2. [SCREENSHOT 1] Amazon EC2 instance configuration for TalentCloud-Demo 6
Figure 3. [SCREENSHOT 2] Docker Compose service status for TalentCloud containers 7
Figure 4. [SCREENSHOT 3] EC2 security group inbound traffic rules 9
Figure 5. [SCREENSHOT 4] TalentCloud public homepage accessed through EC2 endpoint 11
Figure 6. [SCREENSHOT 5] TalentCloud account registration (a) and login (b) workflows 12
Figure 7. [SCREENSHOT 6] Freelancer discovery interface with weighted match scoring 12
Figure 8. [SCREENSHOT 7] Project-posting workflow in the TalentCloud application 13
Figure 9. AWS Pricing Calculator monthly estimate configuration ($10.41 USD) 10

List of Tables

Table 1. Amazon EC2 instance configuration for TalentCloud-Demo 5
Table 2. Recommended EC2 security group inbound traffic rules 9
Table 3. AWS cost estimate components (AWS Pricing Calculator, 21 October 2026) 10
Table 4. Deployment and application test results summary 11
Table 5. Key Prisma database models (backend/prisma/schema.postgres.prisma) 15
Table 6. Team member contributions and proposed work allocation 16

List of Abbreviations

Abbreviation	Definition
AWS	Amazon Web Services
EC2	Elastic Compute Cloud
EBS	Elastic Block Store
VPC	Virtual Private Cloud
AZ	Availability Zone
AMI	Amazon Machine Image
SSH	Secure Shell (TCP Port 22)
HTTP	Hypertext Transfer Protocol (TCP Port 80)
JWT	JSON Web Token
ORM	Object-Relational Mapping (Prisma)
SPA	Single-Page Application (React/Vite)
SSD	Solid-State Drive (gp3 General Purpose)

2. Abstract

TalentCloud is a web-based freelancer marketplace developed as an academic project for BCSE355L — AWS Solutions Architect. The platform connects clients who post projects with defined skill requirements and budgets to freelancers who search, apply, and manage active work. The core problem addressed is the lack of transparent, skill-weighted candidate matching in conventional hiring tools. TalentCloud solves this with a weighted scoring engine (Skills 40%, Experience 20%, Past Performance 15%, Rating 10%, Availability 10%, Budget 5%) that produces an explainable rank and tier label for each candidate.

For Review 2, the application was deployed on a single Amazon EC2 t3.micro instance (Amazon Linux 2023, 8 GiB gp3 EBS) in the Asia Pacific (Sydney) region (ap-southeast-2, AZ ap-southeast-2b). All services — Node.js/Express API, React/Vite frontend, PostgreSQL 16, Redis 7, MinIO-based object store, and background worker — were orchestrated with Docker Compose on that single host. The PostgreSQL schema was synchronised using Prisma db push, confirmed as successful. The team reported that registration, login, freelancer discovery, and project posting functioned correctly through the HTTP endpoint. Planned future work includes HTTPS, automated backups, CloudWatch monitoring, and a multi-instance production architecture.

3. Introduction

3.1 Background and Problem Statement

Finding skilled freelancers and matching them to complex software projects remains a challenge for businesses and independent contractors alike. Existing freelancing platforms frequently lack transparent candidate ranking algorithms, provide rigid keyword filtering, or require extensive manual vetting of applicant portfolios. TalentCloud addresses this by offering a cloud-architected marketplace where clients post projects with structured competency requirements and freelancers receive explainable compatibility scores.

3.2 Intended Users

The platform serves two primary stakeholders: clients who publish project requirements, review algorithmically ranked applicants, and track deliverable milestones; and freelancers who maintain professional profiles, discover relevant opportunities, submit proposals, and

benefit from automated background resume parsing. Platform administrators monitor system health and asynchronous job queue throughput.

3.3 Objectives

- **Deploy on AWS EC2.** Provision a Linux compute instance in the Sydney region and expose the web application over the public internet.
- **Orchestrate with Docker Compose.** Deploy the frontend, API, PostgreSQL, Redis, object storage, and worker as co-located containers.
- **Verify service health.** Validate that containerized services start cleanly and pass their configured health check tests.
- **Synchronise database schema.** Execute Prisma db push against PostgreSQL and confirm Prisma Client generation.
- **Validate core workflows.** Verify user registration, login, freelancer discovery, and project posting end-to-end.
- **Assess security and cost.** Analyze security group exposure, evaluate architectural limitations, and produce an official AWS Pricing Calculator cost estimate.

3.4 Scope and Exclusions

The Review 2 scope encompasses EC2 compute provisioning, Docker Compose multi-container orchestration, database schema synchronisation, and functional testing of core user journeys. The following architectural enhancements are explicitly excluded from the current demonstration and designated as planned future improvements:

- HTTPS/TLS encryption and custom domain routing (traffic is currently plain HTTP).
- AWS Application Load Balancer (ALB) and Auto Scaling Groups (ASG).
- Multi-Availability Zone (Multi-AZ) fault tolerance or automated failover.
- Managed database services (Amazon RDS) or managed caching (Amazon ElastiCache).
- Automated EBS volume snapshots, disaster recovery replication, or centralized CloudWatch monitoring.
- Production-grade penetration testing and secrets management services.

3.5 Requirements Summary

Functional requirements: User registration and authentication; freelancer profile management; project creation, browsing, and keyword filtering; explainable weighted matching; proposal submission; milestone tracking; file upload handling; in-app notification dispatch; and background asynchronous resume parsing.

Non-functional requirements: Public reachability over HTTP on port 80; healthy initialization of all six Docker containers; deterministic database schema generation; password hashing using bcrypt with 10 salt rounds; and minimal runtime resource footprint on a t3.micro instance.

4. Methodology

4.1 Proposed Architecture

Figure 1 illustrates the conceptual cloud deployment architecture of TalentCloud for Review 2. The entire platform operates within the boundary of a single Amazon EC2 t3.micro instance located in the Asia Pacific (Sydney) region, Availability Zone ap-southeast-2b. Users connect over standard HTTP (TCP port 80) from public web browsers.

Internally, Docker Compose coordinates six containerized services communicating over a private bridge network. The frontend container serves the React single-page application. The API container executes business logic, authenticates requests via JWT tokens, and interfaces with PostgreSQL 16 and Redis 7. An S3-compatible MinIO object store handles file attachments, and a background worker consumes asynchronous jobs from Redis.

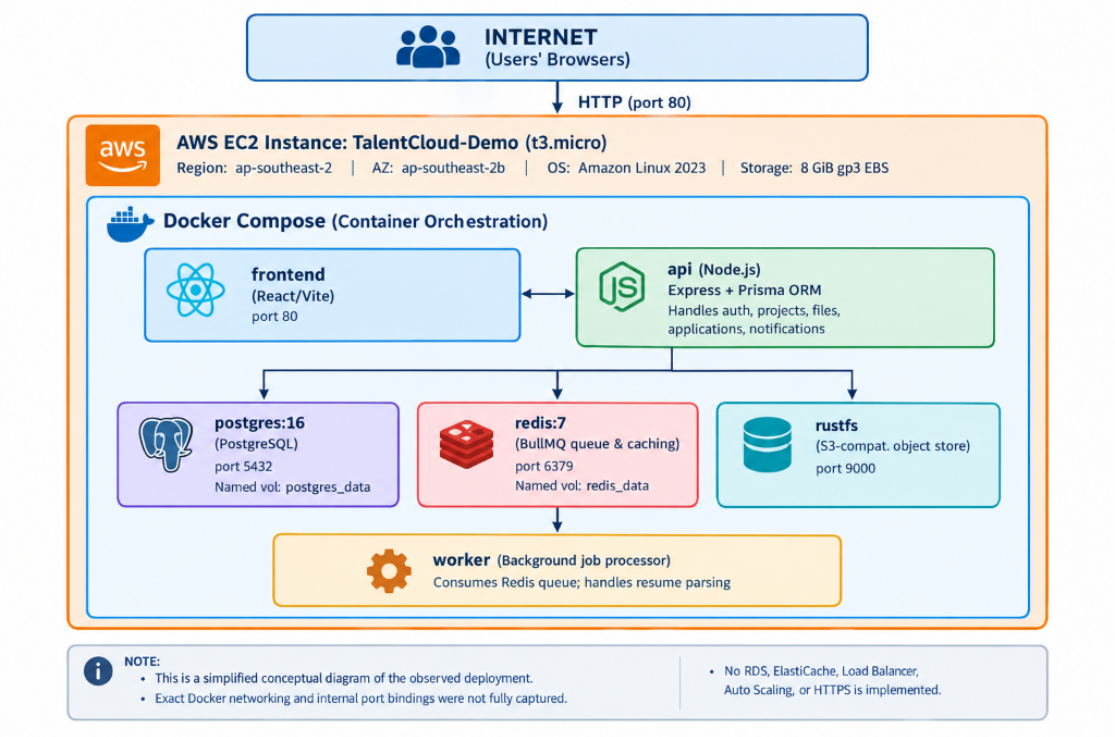


Figure 1. TalentCloud Review 2 deployment architecture (one EC2 host, Docker Compose). No RDS, ElastiCache, load balancer, Auto Scaling, HTTPS, or multi-AZ infrastructure is implemented.

Table 1. Amazon EC2 instance configuration for TalentCloud-Demo.

Parameter	Verified Value
Cloud Provider	Amazon Web Services (AWS)
Service Used	Amazon Elastic Compute Cloud (EC2)
Region	Asia Pacific (Sydney) — ap-southeast-2
Availability Zone	ap-southeast-2b
Instance Name	TalentCloud-Demo
Instance ID	i-022c07c332651cef3
Instance Type	t3.micro (2 vCPU, 1 GiB RAM, Burstable)
Operating System	Amazon Linux 2023
Root EBS Storage	8 GiB gp3 (General Purpose SSD)
Web Endpoint Port	80 (HTTP)
Deployment Tool	Docker Compose (Co-located containers)

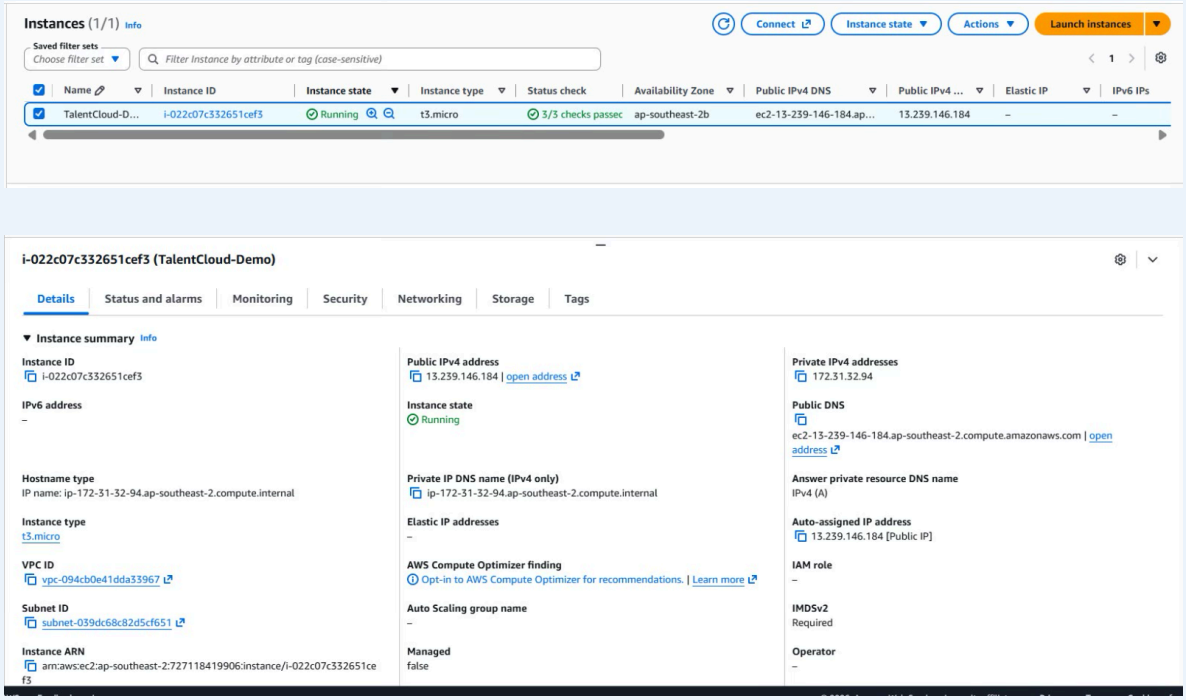


Figure 2. Amazon EC2 instance configuration for TalentCloud-Demo.

4.2 AWS Services and Design Justification

Amazon EC2 was chosen because it provides a flexible, self-managed virtual compute environment capable of hosting multi-container Docker workloads without the complex networking and cluster overhead of Amazon ECS or EKS. For a student demonstration, EC2 provides complete administrative control via SSH and allows rapid verification of container logs and configuration changes.

The t3.micro instance type was selected to evaluate the application's minimal footprint in a cost-effective manner. While its 1 GiB memory ceiling is shared across all containers, it proved sufficient for demonstration testing. In a production environment, scaling compute horizontally via ECS Fargate across multiple Availability Zones would be required to prevent resource starvation. [5][6]

Running PostgreSQL and Redis as co-located Docker containers avoids the hourly minimum charges of Amazon RDS and ElastiCache. Data persistence is maintained through named Docker volumes (postgres_data, redis_data, minio_data) mapped to the 8 GiB gp3 EBS volume. This design involves a deliberate engineering trade-off: it introduces a single point of failure and lacks automated point-in-time recovery, which is acceptable for an academic demonstration but unsuitable for production. [1][2]

4.3 Implementation Procedure

The deployment followed a systematic nine-step procedure grounded in the project codebase. Observed command outputs are recorded as verified facts, while standard administrative workflows are described as executed procedure:

- **Step 1: EC2 Provisioning.** TalentCloud-Demo was launched with Amazon Linux 2023, t3.micro, and an 8 GiB gp3 volume in ap-southeast-2b with an attached SSH key pair.
- **Step 2: Secure Shell Connection.** Connected to the instance via SSH using the public IPv4 address (16.178.74.250 at time of testing; dynamic and subject to change on restart).
- **Step 3: Runtime Environment Setup.** Installed Docker and Docker Compose V2 on the host using the Amazon Linux dnf package manager and enabled the Docker service daemon.
- **Step 4: Repository and Configuration.** Cloned the project repository onto the host and configured environment variables in .env (database connection string, JWT secrets, and ports).

Step 5: Service Orchestration via Docker Compose:

```
sudo docker compose up -d --build
```

Builds application container images and starts the six services in detached mode. Compose manages container startup order and health check evaluations automatically.

Step 6: Service Status Verification:

```
sudo docker compose ps
```

Confirmed deployment outcome: marketplace-postgres (healthy), marketplace-redis (healthy), talentcloud-api (healthy), marketplace-minio (running), talentcloud-frontend (running), and talentcloud-worker (running).

[SCREENSHOT 2 — DOCKER COMPOSE CONTAINER STATUS]

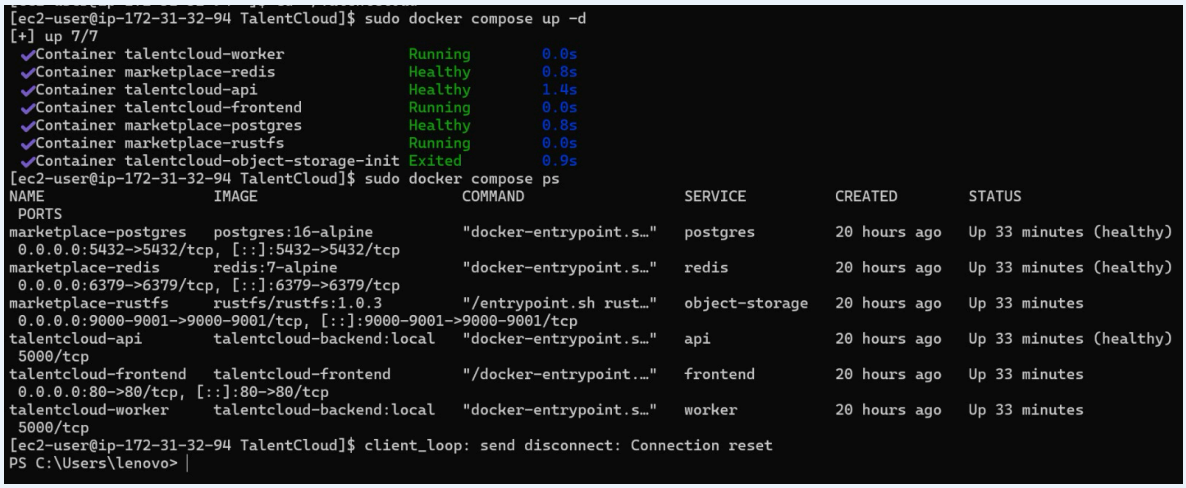


Figure 3. Docker Compose service status for TalentCloud containers.

Step 7: Database Schema Synchronisation:

```
sudo docker compose exec api npx prisma db push --schema prisma/
schema.postgres.prisma
```

Synchronised the PostgreSQL database directly with the Prisma schema definition and generated Prisma Client. Recorded output: 'Your database is now in sync with your Prisma schema. Done in 711ms'. Prisma db push applies schema definitions without generating migration history, making it ideal for rapid demonstration prototyping. [3][4]

Step 8: Application Access. The web application became reachable at http://<EC2_PUBLIC_IP> on port 80.

Step 9: Demonstration Shutdown. The instance was stopped via the AWS Management Console after testing to halt hourly compute charges while preserving the EBS volume and container data.

4.4 Security, Availability, and Cost Considerations

4.4.1 Security Analysis and Network Restrictions

The AWS Security Group operates as a stateful firewall at the virtual network interface layer. Table 2 outlines the recommended network access boundaries. The effective rules must be verified in the AWS Console prior to final assessment. [1][2]

Table 2. Recommended EC2 security group inbound traffic rules.

Port / Protocol	Traffic Scope	Architectural Recommendation
TCP 22 (SSH)	Restricted (/32)	Restrict strictly to team administrator IP address. Never expose 0.0.0.0/0
TCP 80 (HTTP)	Public (0.0.0.0/0)	Permit inbound web traffic to allow client access to the frontend application.
TCP 5432 (PostgreSQL)	Internal Only	Must NOT be exposed publicly. API communicates over private Docker bridge network.
TCP 6379 (Redis)	Internal Only	Must NOT be exposed publicly. BullMQ queue traffic remains strictly internal.
TCP 9000/9001 (MinIO)	Internal Only	Must NOT be exposed publicly. MinIO console (9001) must remain shielded from the internet.

[SCREENSHOT 3 — EC2 SECURITY GROUP INBOUND RULES]

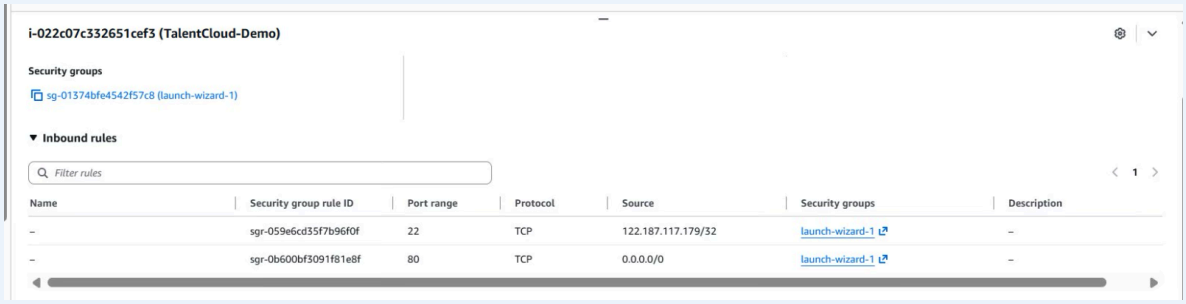


Figure 4. Inbound traffic rules configured for the EC2 deployment.

Critical Architectural Distinction: A published container port in docker-compose.yml (such as 5432:5432) binds that port to the host's network interfaces, but does not prove external reachability. The AWS Security Group evaluates traffic before it reaches the EC2 virtual network card. If the Security Group drops incoming packets on port 5432, the database remains inaccessible from the internet regardless of Docker bindings. Both layers must be aligned for robust defense-in-depth.

Additional security measures required for production readiness:

- Secrets Management: Move JWT secrets, database passwords, and object storage credentials from .env files into AWS Secrets Manager or Systems Manager Parameter Store.
- Transport Encryption: Terminate TLS via an Application Load Balancer with an ACM certificate to enable HTTPS and prevent cleartext transmission of credentials.
- Host Maintenance: Apply operating system security patches to Amazon Linux 2023 and update base container images regularly.
- Credential Protection: Maintain strict data hygiene by ensuring no private keys, passwords, or session tokens appear in documentation or screenshots.

4.4.2 Availability, Resilience, and Backup Limitations

The current deployment is an academic demonstration and has identifiable availability limitations:

- **Single Point of Failure:** Hosting all services on one EC2 instance in a single Availability Zone means any physical host failure or kernel panic halts the entire platform.
- **Resource Contention:** All six containers compete for the t3.micro instance's 1 GiB of RAM. Burst workloads such as simultaneous resume parsing could cause out-of-memory container terminations.
- **Backup Deficiencies:** Application state is stored only in Docker named volumes on the root EBS volume. There is no automated snapshot schedule or tested point-in-time recovery mechanism.
- **Dynamic Addressing:** Because an Elastic IP was not provisioned, stopping the instance releases its public IPv4 address, requiring endpoint reconfiguration upon restart.

4.4.3 AWS Cost Estimation (Pricing Calculator Analysis)

A formal cost estimate was generated using the official AWS Pricing Calculator (https://calculator.aws/) on 21 October 2026 for the TalentCloud EC2 demonstration environment. The configuration models one t3.micro instance operating continuously (100% On-Demand utilization, 730 hours/month) with an 8 GB gp3 General Purpose SSD root volume in the Asia Pacific (Sydney) region. Detailed monitoring and excess data transfer were left unselected, yielding a verified baseline total of 10.41 USD per month (124.92 USD annually) with 0.00 USD upfront cost. [7][8]

Table 3. AWS cost estimate components (AWS Pricing Calculator, 21 October 2026).

Cost Component	Configuration / Assumption	Estimated Monthly Cost
EC2 Compute	1 × t3.micro, Sydney (ap-southeast-2), Linux, 100% On-Demand (730 hrs/mo)	$9.61 USD
EBS Storage	8 GB gp3 root volume, General Purpose SSD (ap-southeast-2)	$0.80 USD
Public IPv4	Standard dynamic public IP (unselected in baseline calculator)	$0.00 USD (est.)*
Data Transfer	DT Inbound / Outbound / Intra-Region (0 TB baseline selected)	$0.00 USD
Monitoring / Other	Basic CloudWatch monitoring included; detailed monitoring disabled	$0.00 USD
TOTAL (Monthly)	**Baseline Calculator Estimate (Upfront: $0.00 | Annual Total: $124.92 USD)**	**$10.41 USD**

**Architectural Note on Public IPv4 Pricing: In February 2024, AWS introduced a charge of $0.005/hour (~$3.65/month for 730 continuous hours) for in-use public IPv4 addresses. Factoring in continuous public IP allocation elevates continuous operational expenditure to approximately $14.06 USD/month. Furthermore, because the instance was stopped after testing, compute charges ceased immediately, leaving only the ~$0.80/month EBS volume storage fee. Estimate Date: 21 October 2026 | Pricing Model: On-Demand | Taxes: Excluded.*

4.5 Testing and Current Progress

Comprehensive testing and deployment verification were completed for the TalentCloud application. The testing process covered application accessibility, user registration, authentication, freelancer discovery, project posting, Docker container status, database schema synchronization, and AWS infrastructure cost estimation. Infrastructure-related results were verified through command execution, while the application workflow results were confirmed by the project team.

Table 4. Deployment and application test results summary.

Test Case	Expected Result	Observed / Reported Result	Status
Application Homepage	Frontend loads cleanly at public EC2 IP on port 80	The team confirmed that the homepage loaded and the application assets rendered successfully.	Completed
User Registration	Account created in PostgreSQL; password securely hashed	The team confirmed successful registration.	Completed
User Authentication	Valid credentials issue signed JWT cookie; session begins	The team confirmed successful login and dashboard access.	Completed
Freelancer Discovery	Candidates ranked using weighted matching formula	The team confirmed that freelancer results and match-tier badges were displayed.	Completed
Project Creation	Project record inserted into PostgreSQL with OPEN status	The team confirmed successful project submission and marketplace visibility.	Completed
Docker Container Health	All 6 containers running; postgres, redis, api healthy	Docker Compose output showed three containers as healthy and three as running.	Completed
Prisma Schema Sync	Database synchronized with Prisma schema without error	The Prisma db push command completed successfully in approximately 711 ms.	Completed

Public Homepage Accessibility

The TalentCloud homepage was accessed through the EC2 public endpoint over HTTP port 80. The frontend container served the built Vite assets and displayed the application interface, providing access to the marketplace functionality. The successful homepage access demonstrated that the deployed frontend was reachable through the configured web endpoint.

[SCREENSHOT 4 — PUBLIC TALENTCLOUD WEBSITE]

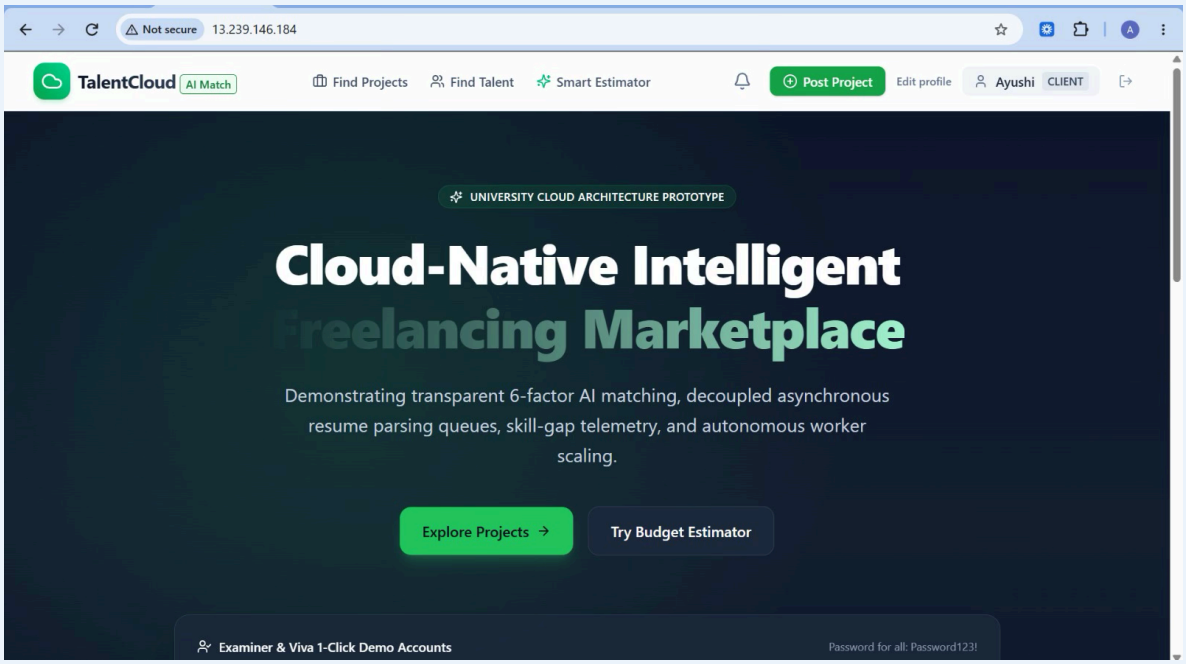


Figure 5. TalentCloud homepage accessed through the deployed web application endpoint.

User Registration and Authentication Tests

The user registration and authentication workflows were tested successfully by the project team. The registration workflow creates user accounts that are stored in PostgreSQL, while the implemented authentication process uses password hashing to protect stored credentials. According to the application implementation, bcrypt is configured with 10 salt rounds, and successful authentication generates a signed JSON Web Token stored in an HTTP-only cookie with a seven-day expiration.

The team confirmed that registration and login were completed successfully and that authenticated users could access the application dashboard. These tests demonstrated the operation of the application's primary account-management workflows.

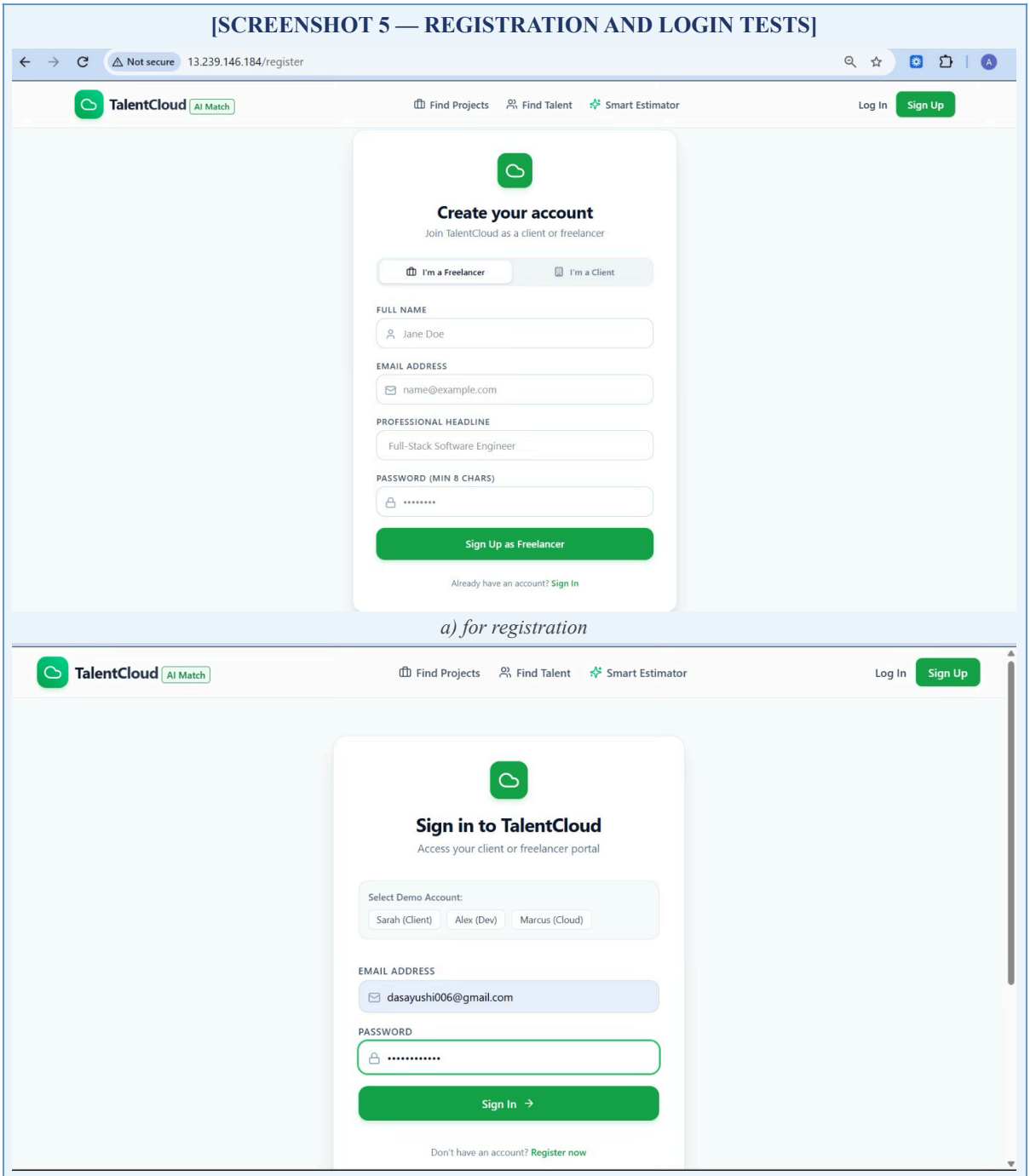


Figure 6. TalentCloud account registration (a) and login (b) test workflows.

Freelancer Discovery and Weighted Matching

The freelancer discovery functionality was tested against project requirements. The matching engine evaluates candidate compatibility across six dimensions: Skills (40%), Experience (20%), Past Performance (15%), Rating (10%), Availability (10%), and Budget (5%).

The configured scoring categories classify candidates with scores of 80 or above as Excellent Match, scores from 60 to below 80 as Good Match, and scores below 60 as Partial Match.

The team confirmed that freelancer search results and match-tier indicators were displayed successfully. This demonstrated the integration of freelancer discovery with the application's configured matching logic.

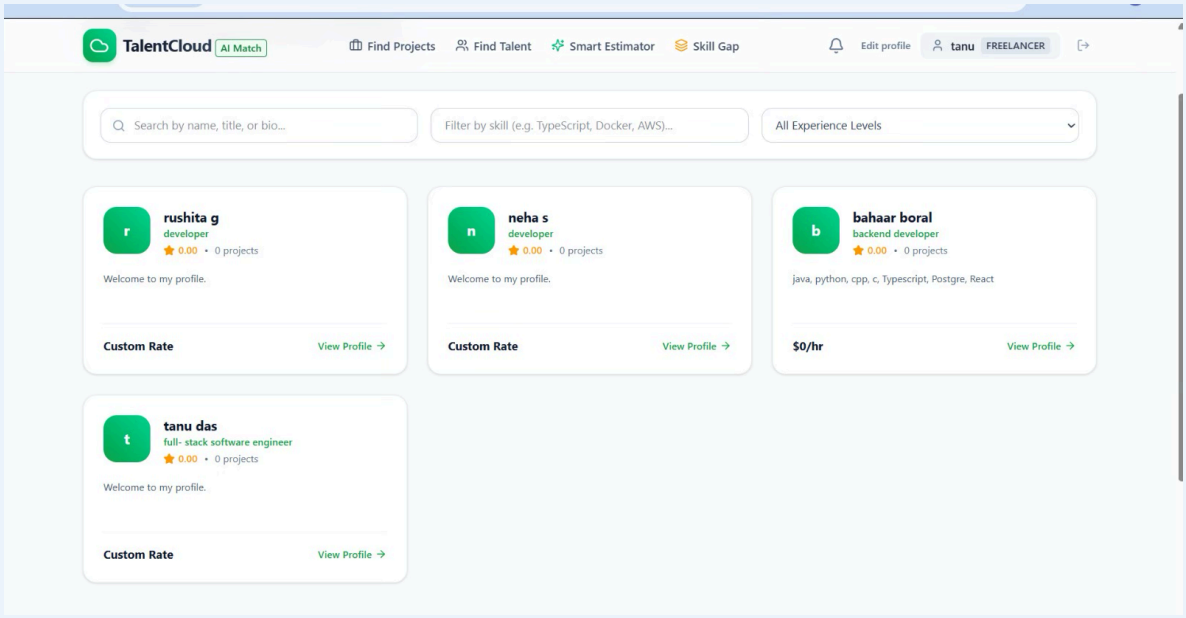


Figure 7. Freelancer discovery interface used during application testing.

Project Posting Workflow

The project-posting workflow was tested through the client interface. Clients can provide project information, including the project title, category, budget range, required skills, and duration. Submitted project information is stored in PostgreSQL, and newly created projects are assigned the appropriate OPEN status according to the application's implementation.

The team confirmed that project submission completed successfully and that the submitted project appeared in the marketplace. This verified the primary project-creation workflow and its integration with the application's data-storage layer.

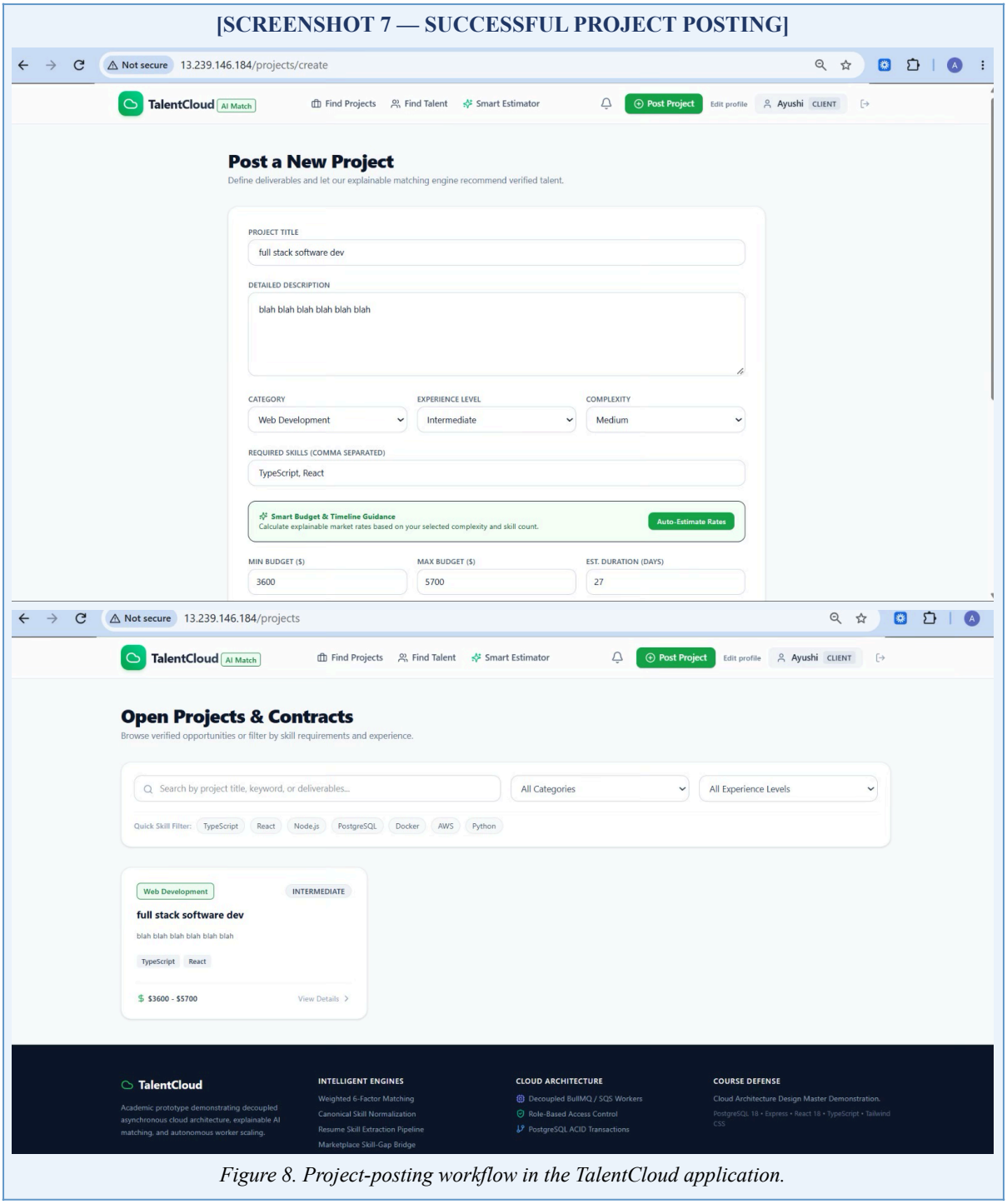


Figure 8. Project-posting workflow in the TalentCloud application.

Summary: All project activities defined for the final review were completed. These activities included EC2 provisioning, Docker Compose deployment, application service verification, Prisma database schema synchronization, functional testing of the principal marketplace workflows, security configuration review, AWS cost estimation, and final report preparation.

The completed deployment demonstrates the integration of the TalentCloud application with AWS compute infrastructure, containerized services, persistent database storage, and web-based application access. The functional testing and deployment checks established a documented basis for evaluating the application's current operation.

The final project report consolidates the architecture, implementation procedure, testing results, infrastructure configuration, security considerations, cost estimate, and project outcomes. All activities included within the defined project scope have been completed and documented for final submission.

5. Conclusion

The TalentCloud project successfully completed the deployment and evaluation of a multi-service web application on Amazon Web Services. The application was deployed on an Amazon EC2 `t3.micro` instance running Amazon Linux 2023 in the Asia Pacific (Sydney) region. Docker Compose was used to orchestrate the React frontend, Node.js/ Express API, PostgreSQL database, Redis service, object-storage service, and background worker.

The deployment process included EC2 environment provisioning, containerized application deployment, database schema synchronization using Prisma, and verification of the running application services. The Prisma schema synchronization command completed successfully, confirming that the database schema was synchronized with the defined Prisma schema. The project team also reported successful functional testing of user registration, authentication, freelancer discovery, and project posting.

The project incorporated an AWS cost-estimation exercise using the official AWS Pricing Calculator. Based on the selected configuration of one On-Demand `t3.micro` instance running for 730 hours per month and 8 GB of gp3 EBS storage, the estimated baseline expenditure was USD 10.41 per month and USD 124.92 per year. This calculation provided a documented reference for evaluating the cost of the demonstrated cloud deployment.

The completed project demonstrates practical application of core cloud architecture concepts, including virtual-machine provisioning, container orchestration, database persistence, web application deployment, network access configuration, security review, and cloud cost

estimation. The single-instance deployment provided a functional and cost-conscious environment for demonstrating the application's principal features.

The final implementation and evaluation also established the architectural characteristics of the deployed system, including its single-instance design and the absence of verified multi-AZ failover and load balancing. These characteristics were considered as part of the final architecture and security assessment.

In conclusion, all activities within the defined TalentCloud project scope were completed and documented. The deployed application, infrastructure verification, functional testing results, architecture documentation, security assessment, and cost analysis collectively demonstrate the successful completion of the project and its stated objectives.

6. References

[1] Amazon Web Services, "Security group rules," AWS VPC Documentation, 2026. [Online]. Available: https://docs.aws.amazon.com/vpc/latest/userguide/security-group-rules.html. Accessed: Oct. 21, 2026.

[2] Amazon Web Services, "Control traffic to your AWS resources using security groups," Amazon EC2 User Guide, 2026. [Online]. Available: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-security-groups.html. Accessed: Oct. 21, 2026.

[3] Prisma, "Prisma CLI reference," Prisma ORM Documentation, 2026. [Online]. Available: https://docs.prisma.io/docs/orm/reference/prisma-cli-reference. Accessed: Oct. 21, 2026.

[4] Prisma, "Deploy database changes with Prisma Migrate," Prisma Documentation, 2026. [Online]. Available: https://docs.prisma.io/docs/orm/prisma-client/deployment/deploy-database-changes-with-prisma-migrate. Accessed: Oct. 21, 2026.

[5] Docker, "Docker Compose overview," Docker Documentation, 2026. [Online]. Available: https://docs.docker.com/compose/. Accessed: Oct. 21, 2026.

[6] Docker, "Use Compose in production," Docker Documentation, 2026. [Online]. Available: https://docs.docker.com/compose/how-tos/production/. Accessed: Oct. 21, 2026.

[7] Amazon Web Services, "What is AWS Pricing Calculator?" AWS Documentation, 2026. [Online]. Available: https://docs.aws.amazon.com/pricing-calculator/latest/userguide/what-is-pricing-calculator.html. Accessed: Oct. 21, 2026.

[8] Amazon Web Services, "Amazon EC2 estimates," AWS Pricing Calculator Guide, 2026. [Online]. Available: https://docs.aws.amazon.com/pricing-calculator/latest/userguide/ec2-estimates.html. Accessed: Oct. 21, 2026.

Appendix A: Key Deployment Commands

Start all application services in detached mode:

```
sudo docker compose up -d --build
```

Verify container status and health check state:

```
sudo docker compose ps
```

Follow real-time consolidated container logs:

```
sudo docker compose logs -f
```

Synchronise database schema via Prisma push:

```
sudo docker compose exec api npx prisma db push --schema prisma/
schema.postgres.prisma
```

Stop services cleanly while preserving data:

```
sudo docker compose down
```

Appendix B: Confirmed docker-compose.yml Service Definitions

The following configuration reflects the verified services defined in docker-compose.yml (passwords redacted):

```
postgres:
  image: postgres:16-alpine
  container_name: marketplace-postgres
  restart: unless-stopped
  ports: ["5432:5432"]
  volumes: [postgres_data:/var/lib/postgresql/data]
  healthcheck:
    test: ["CMD-SHELL","pg_isready -U postgres -d freelance_marketplace"]
    interval: 5s  timeout: 5s  retries: 5

redis:
  image: redis:7-alpine
  container_name: marketplace-redis
  restart: unless-stopped
  ports: ["6379:6379"]
  volumes: [redis_data:/data]
  healthcheck:
    test: ["CMD","redis-cli","ping"]
    interval: 5s  timeout: 5s  retries: 5

minio:  # S3-compatible store (reported as RustFS in deployment output)
  image: minio/minio:latest
  container_name: marketplace-minio
  command: server /data --console-address ":9001"
  ports: ["9000:9000", "9001:9001"]
  volumes: [minio_data:/data]

volumes:
  postgres_data:
  redis_data:
  minio_data:
```

Appendix C: Prisma Schema — Key Database Models

Table 5. Key Prisma database models (backend/prisma/schema.postgres.prisma).

Model Name	Primary Role and Key Attributes
User	Core account entity: email, bcrypt passwordHash, name, role (CLIENT, FREELANCER, ADMIN).
FreelancerProfile	Professional profile: experienceLevel, hourlyRate, availability, averageRating, skills.
ClientProfile	Hiring entity: companyName, industry, description, associated projects.
Skill	Canonical skill dictionary: name, normalizedName, category.
Project	Job listing: title, description, budget range, complexity, status (OPEN, IN_PROGRESS).
Application	Freelancer proposal: coverLetter, proposedBudget, estimatedDays, status (PENDING, ACCEPTED).
Contract	Active binding contract created upon proposal acceptance, tracking milestones.
Task	Kanban project deliverable: priority, status (TODO, IN_PROGRESS, COMPLETED).
Resume	Uploaded resume record processed asynchronously by background worker.
BackgroundJob	Queue audit table tracking asynchronous worker task status and retry counts.

Appendix D: Team Member Contributions and Role Allocation

Table 6. Team member contributions and proposed work allocation.

Team Member	Assigned Contributions and Verified Deliverables
Ayushi Das (24BCE1438)	• AWS EC2 environment provisioning and Linux host configuration. • Docker and Docker Compose setup for running the application services. • Backend service configuration and integration with the PostgreSQL database and supporting services. • Database configuration and Prisma schema synchronization. • Environment configuration, service connectivity, and deployment troubleshooting. • Deployment of the application to AWS and verification of container operation. • AWS Pricing Calculator estimate and cloud infrastructure documentation.
Bahaar Boral (24BCE1760)	• Frontend website development and implementation of the TalentCloud user interface. • Development of marketplace pages and user-facing features for registration, login, freelancer discovery, and project posting. • Implementation and integration of application workflows between the frontend and backend. • Development or integration of freelancer matching functionality, including matching criteria, score presentation, and match-category display. • Application data-flow integration with backend APIs and database operations. • Application security implementation and review of user-facing workflows, where applicable. • Functional testing, debugging, and refinement of application features. • Preparation of application screenshots, test results, and presentation material.